Up to now…

- We know how a server can send static HTML pages or HTML pages in which it injects data before sending the response to a browser
- We know how a server can send data (e.g. JSON) to a browser that can display the «raw» data
- What is missing?

  We would like (why???) to send a view and modify it on-the-fly with data sent later on (do you know Single Page Apps?)

1

UNIVERSITÀ DI TRENTO

How can we avoid unresponsiveness of pages ?

2

Responsivness

- Responsivness *"refers to the specific ability of a system or functional unit to complete assigned tasks within a given time"* (https://en.wikipedia.org/wiki/Responsiveness)
- A long delay in the system's response can, indeed, cause user frustration. Several empirical tolerance thresholds can be identified for the web:

Waiting time	Perception
0.1s	The response is perceived as instantaneous
1s	A slight delay is perceived, but it is ok for website
10s	There is an impact on user productivity

3

Responsivness is not responsive

Responsivness is different from responsive design

Responsive design is aimed at having a good rendering of web pages on a variety of devices / screen to ensure usability and satisfaction

Responsive design is a matter of HTML and CSS

4

First historical solution: Ajax

- Ajax is not a technology in itself: it is a term coined in 2005 by Jesse James Garrett: “Asynchronous JavaScript + XML”.
- It is a technique:
  - to blur the line between web-based and desktop applications
  - rich, highly responsive and useful for interactive interfaces

Ajax was born as:

- dynamic presentation based on XHTML + CSS;
- dynamic display and interaction using DOM;
- data exchange and manipulation using XML e XSLT;
- asynchrounous data fetching using XMLHttpRequest;
- JavaScript as glue.

- All major browsers still support Ajax (there are milions of web apps using it!). Today, other solutions exist that are less verbose (see later on)

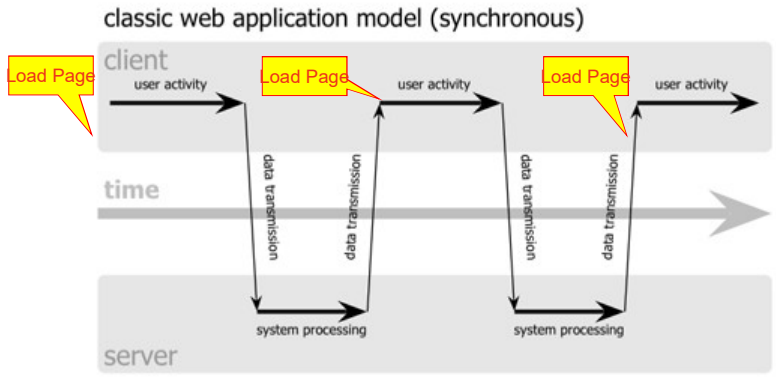
5

How does Ajax work?

- The core idea behind AJAX is to make the communication with the server asynchronous, so that data is transferred and processed in the background
- As a result the user can continue working on the other parts of the page without interruption
- In an AJAX-enabled application only the relevant page elements are updated, only when this is necessary.

6

Changing paradigm

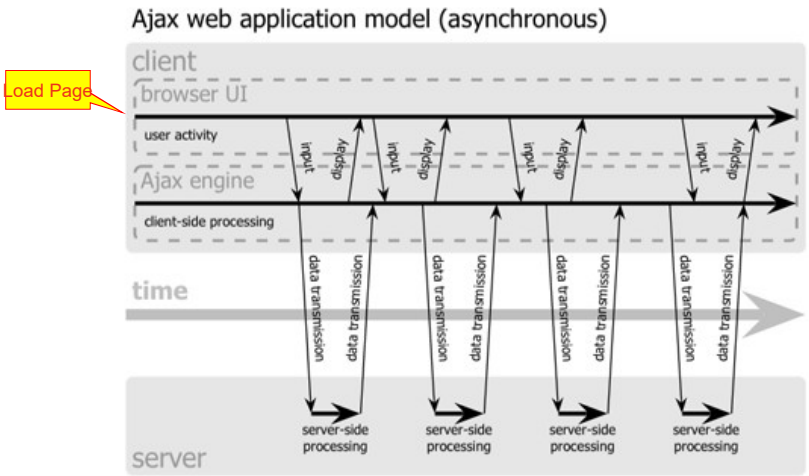


Pictures after Jesse James Garrett

Work-wait

7

Changing paradigm



Pictures after Jesse James Garrett

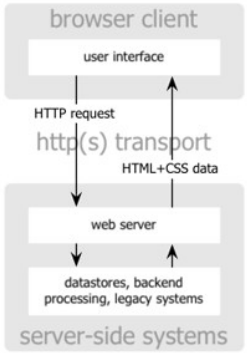
Work-work

8

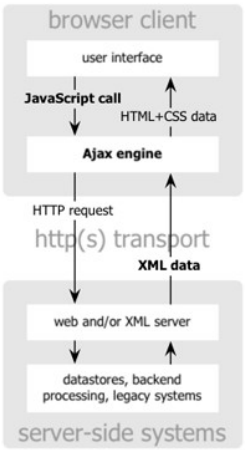
The paradigms

Pictures after
Jesse James Garrett

1.0



classic
web application model

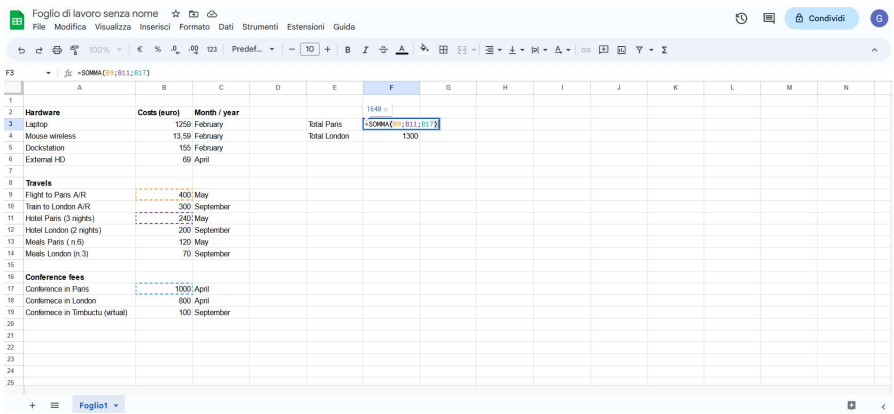


2.0

Ajax
web application model

9

The (impressive!) result



10

Ajax - advantages

- Better Performance and Efficiency
  - small amount of data transferred from the server. Beneficial for data-intensive applications as well as for low-bandwidth networks
- More Responsive Interfaces
  - the improved performance give the feeling that updates are happening instantly. AJAX web applications appear to behave much like their desktop counterparts
- Reduced or Eliminated "Waiting" Time
  - only the relevant page elements are updates, with the rest of the page remaining unchanged. This decreases the idle waiting time
- Increased Usability
  - Users can work with the rest of the page while data is being transferred in the background

11

XMLHttpRequest (XHR)

see https://developer.mozilla.org/en-US/docs/Web/API/XMLHttpRequest

```
function loadDoc() {
  var xhttp = new XMLHttpRequest();
  xhttp.onreadystatechange = function() {
    if (this.readyState == 4) {
         if(this.status == 200){
              document.getElementById("demo").textContent =
              this.responseText;}
         else
             document.getElementById("demo").textContent =
              "Error!";}
  };
  xhttp.open("GET", "ajax_info.txt", true);
  xhttp.send();
}
```

Asynchronous Programming!

12

XMLHttpRequest properties

Property	Description
onreadystatechange	Defines a function to be called when the readyState property changes
readyState	Holds the status of the XMLHttpRequest. 0: request not initialized 1: server connection established 2: request received 3: processing request 4: request finished and response is ready
responseText	Returns the response data as a string
responseXML	Returns the response data as XML data
status	Returns the HTTP status-number of a request, e.g. 200: "OK" 403: "Forbidden" 404: "Not Found"
statusText	Returns the status-text (e.g. "OK" or "Not Found")

13

XMLHttpRequest.readyState

The **XMLHttpRequest.readyState** property returns the state an XMLHttpRequest client is in. An XHR client exists in one of the following states:

Value	State	Description
0	UNSENT	Client has been created. open() not called yet.
1	OPENED	open() has been called.
2	HEADERS_RECEIVED	send() has been called, and headers and status are available.
3	LOADING	Downloading; responseText hol ds partial data.
4	DONE	The operation is complete.

14

XMLHttpRequest methods

new XMLHttpRequest()	Creates a new XMLHttpRequest object
abort()	Cancels the current request
getAllResponseHeaders()	Returns header information
getResponseHeader()	Returns specific header information
open(*method, url, async, user, psw*)	Specifies the request *method*: the request type **GET** or **POST** *url*: the file location *async*: true (asynchronous) or false (synchronous) *user*: optional user name *psw*: optional password
send()	Sends the request to the server Used for GET requests
send(*string*)	Sends the request to the server. Used for POST requests
setRequestHeader()	Adds a label/value pair to the header to be sent

15

XMLHttpRequest – Getting static resources

```
function loadDoc() {
  var xhttp = new XMLHttpRequest();
  xhttp.onreadystatechange = function() {
    if (this.readyState == 4) {

        if(this.status == 200){

      document.getElementById("demo").textContent =
            this.responseText;}
         else

document.getElementById("demo").textContent =
            "Error!";}
  };
  xhttp.open("GET", "ajax_info.txt", true);
  xhttp.send();
}
```

16

Getting dynamic resources with GET

```
function loadDoc() {
  var xhttp = new XMLHttpRequest();
  xhttp.onreadystatechange = function() {
    if (this.readyState == 4) {
         if(this.status == 200){
              document.getElementById("demo").textContent =
              this.responseText;}
         else
             document.getElementById("demo").textContent =
              "Error!";}
  };
  xhttp.open("GET", "mything?param1=27", true);
  xhttp.send();
}
```

If you want to avoid getting cashed results, add a fake parameter with the current time, e.g.:

```
xhttp.open("GET", url + ((/\?/).test(url) ? "&" : "?") + (new Date()).getTime());
```

See https://www.w3schools.com/jsref/jsref_regexp_test.asp to understand the code above

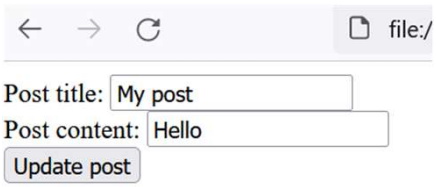
17

Getting dynamic resources with POST

```
function loadDoc() {
  var xhttp = new XMLHttpRequest();
  xhttp.onreadystatechange = function() {
    if (this.readyState == 4) {
         if(this.status==200 || this.status=201){
              document.getElementById("demo").textContent =
              this.responseText;}
         else
             document.getElementById("demo").textContent =
              "Error!";}
  };
  xhttp.open("POST", "/api/utenti/salva", true);
  xhttp.setRequestHeader("Content-type",
  "application/x-www-form-urlencoded");
  xhttp.send("nome=Dorothea&lname=Wierer");
}
```


18

A remark



- In these cases, to what is the click action linked ?
- Do we need to use a button with type = "sumit" or with type = "button"?

19

Make sure that you…

- Preserve the Normal Page Lifecycle – as much as possible!
- Support Cross-Browser usage – there are different implementation of the XmlHttpRequest object. You should make sure that all AJAX components you choose operate properly on various browsers and platforms
- Give visual feedback - When a user clicks on something in the AJAX user interface, they need immediate visual feedback
- Keep the Back button – make sure that the Back button in your application functions on every page of the site

20

Playing with AJAX!
(just once in your life ☺)

demo_AJAX